

UNIVERSITY OF ENGINEERING AND TECHNOLOGY PESHAWAR

Department of Electrical Engineering

LABORATORY REPORT

Subject:	EE-170L Computer Programming Lab
Lab No:	Lab 10
Lab Title:	More About Functions in C++
Semester:	Semester 2
Session:	Spring 2026

Submitted By:

Name: Muhammad Adil

Registration No: BF25NWELE06699

1. OBJECTIVE

To learn and apply function call by value, call by reference, and default parameters in C++. To understand the differences between these parameter passing mechanisms and when to use each.

2. THEORY

This lab covers advanced function concepts in C++:

(1) Call by Value: A copy of the variable is passed to the function. Changes made inside the function do not affect the original variable. This is the default parameter passing mechanism in C++.

(2) Call by Reference: The address (reference) of the variable is passed using the & operator. The function works directly on the original variable. Changes made inside the function affect the original variable permanently.

(3) Default Parameters: Values specified in the function declaration that are used when the corresponding argument is not provided during the function call. Default values must be specified from right to left in the parameter list.

3. LAB TASKS

3.1 Function with Parameters (multiply function)

Declare a function called 'multiply' that takes two integer parameters: 'num1' and 'num2'. Inside the function, calculate the product of the two numbers. Display the result within the function. Call the function from the main program, passing two numbers, to perform the multiplication.

Source Code:

```
#include <iostream>
using namespace std;

// =====
// Function Prototype
// =====
// multiply() - Multiplies two integers and displays result
// Parameters: num1, num2 - integers to be multiplied
// Returns: void (no return value, displays result directly)
void multiply(int num1, int num2);

int main()
{
    // =====
    // Variable Declaration
    // =====
    int number1, number2; // Variables to store user input

    // =====
    // Input Section
```

```

// =====
cout << "Enter first number: ";
cin >> number1; // Read first number from user

cout << "Enter second number: ";
cin >> number2; // Read second number from user

// =====
// Function Call
// =====
cout << "\nCalling multiply function..." << endl;
multiply(number1, number2); // Pass both numbers to function

return 0;
}

// =====
// Function Definition: multiply
// =====
// This function takes two integers, multiplies them,
// and displays the result
// Parameters:
// num1 - first integer
// num2 - second integer
void multiply(int num1, int num2)
{
    int product; // Variable to store multiplication result

    // Calculate product of two numbers
    product = num1 * num2;

    // Display the result
    cout << "=====" << endl;
    cout << " Multiplication Result " << endl;
    cout << " " << num1 << " x " << num2 << " = " << product << endl;
    cout << "=====" << endl;
}

```

Expected Output:

```

Enter first number: 12
Enter second number: 5

```

```

Calling multiply function...
=====
Multiplication Result
12 x 5 = 60
=====

```

3.2 Function with Return Value (getSquare function)

Declare a function called 'getSquare' that takes an integer parameter 'number'. Inside the function, calculate the square of the number. Return the result. Call the function from the main program, passing a number, and display the returned square.

Source Code:

```
#include <iostream>
using namespace std;

// =====
// Function Prototype
// =====
// getSquare() - Calculates square of a number
// Parameter: number - integer to be squared
// Returns: int - the square of the number
int getSquare(int number);

int main()
{
    // =====
    // Variable Declaration
    // =====
    int num;          // Variable to store user input
    int square;       // Variable to store returned square value

    // =====
    // Input Section
    // =====
    cout << "Enter a number to find its square: ";
    cin >> num;  // Read number from user

    // =====
    // Function Call and Store Return Value
    // =====
    square = getSquare(num);  // Call function and capture return value

    // =====
    // Display Result
    // =====
    cout << "\n===== RESULT =====" << endl;
    cout << "Number: " << num << endl;
    cout << "Square: " << square << endl;
    cout << "===== " << endl;

    // Demonstrate with hardcoded values
    cout << "\nSquare of 7 = " << getSquare(7) << endl;
```

```

        cout << "Square of 12 = " << getSquare(12) << endl;

    return 0;
}

// =====
// Function Definition: getSquare
// =====
// This function calculates the square of a given number
// Parameter:
//   number - the integer to be squared
// Returns:
//   The square of the number (number * number)
int getSquare(int number)
{
    // Calculate and return the square
    return (number * number); // Return square to calling function
}

```

Expected Output:

```
Enter a number to find its square: 8
```

```

===== RESULT =====
Number: 8
Square: 64
=====

```

```

Square of 7 = 49
Square of 12 = 144

```

3.3 Demonstration: Call by Value vs Call by Reference

Demonstrate the difference between call by value and call by reference using a swapping example. In call by value, the original variables remain unchanged. In call by reference, the original variables are actually swapped.

Source Code:

```

#include <iostream>
using namespace std;

// =====
// Function Prototypes
// =====
// swapByValue() - Attempts to swap using call by value
// Parameters receive COPIES of original variables
void swapByValue(int a, int b);

```

```

// swapByReference() - Swaps using call by reference (&)
// Parameters receive REFERENCES (addresses) of original variables
void swapByReference(int &a, int &b);

int main()
{
    // =====
    // Variables for Call by Value Demonstration
    // =====
    int x = 50, y = 70;

    cout << "===== CALL BY VALUE =====" << endl;
    cout << "Before swapByValue: x = " << x << ", y = " << y << endl;
    swapByValue(x, y); // Pass copies of x and y
    cout << "After swapByValue: x = " << x << ", y = " << y << endl;
    cout << "(Values remain unchanged!)" << endl << endl;

    // =====
    // Variables for Call by Reference Demonstration
    // =====
    int a = 50, b = 70;

    cout << "===== CALL BY REFERENCE =====" << endl;
    cout << "Before swapByReference: a = " << a << ", b = " << b << endl;
    swapByReference(a, b); // Pass references of a and b
    cout << "After swapByReference: a = " << a << ", b = " << b << endl;
    cout << "(Values are actually swapped!)" << endl;

    return 0;
}

// =====
// Function: swapByValue
// =====
// This function receives COPIES of the variables
// Changes made here do NOT affect original variables
void swapByValue(int a, int b)
{
    int temp; // Temporary variable for swapping

    temp = a; // Store a in temp
    a = b;    // Copy b to a
    b = temp; // Copy temp (original a) to b

    // Display values inside function (only local copies changed)
    cout << "Inside swapByValue: a = " << a << ", b = " << b << endl;
}

```

```

// =====
// Function: swapByReference
// =====
// This function receives REFERENCES (addresses) of variables
// Changes made here DO affect original variables
// & indicates reference parameter
void swapByReference(int &a, int &b)
{
    int temp;    // Temporary variable for swapping

    temp = a;    // Store original a in temp
    a = b;       // Change original a to value of b
    b = temp;    // Change original b to value of temp

    // Display values inside function (original variables changed)
    cout << "Inside swapByReference: a = " << a << ", b = " << b << endl;
}

```

Expected Output:

```

===== CALL BY VALUE =====
Before swapByValue: x = 50, y = 70
Inside swapByValue: a = 70, b = 50
After swapByValue:  x = 50, y = 70
(Values remain unchanged!)

===== CALL BY REFERENCE =====
Before swapByReference: a = 50, b = 70
Inside swapByReference: a = 70, b = 50
After swapByReference:  a = 70, b = 50
(Values are actually swapped!)

```

3.4 Demonstration: Default Parameters

Demonstrate default parameter values in functions. When a function is called without arguments, default values are used. When arguments are provided, they override the defaults.

Source Code:

```

#include <iostream>
using namespace std;

// =====
// Function with Default Parameters
// =====
// divide() - Divides two numbers with default values
// Parameters:
//   a - numerator (default: 8)
//   b - denominator (default: 2)

```

```

// Returns: int - result of division
int divide(int a = 8, int b = 2);

int main()
{
    // =====
    // Test Default Parameters
    // =====

    // Call with no arguments - uses both defaults (8/2)
    cout << "divide() with no arguments:" << endl;
    cout << "divide() = " << divide() << endl << endl;

    // Call with one argument - uses default for second (12/2)
    cout << "divide(12) with one argument:" << endl;
    cout << "divide(12) = " << divide(12) << endl << endl;

    // Call with both arguments - no defaults used (63/7)
    cout << "divide(63, 7) with both arguments:" << endl;
    cout << "divide(63, 7) = " << divide(63, 7) << endl;

    return 0;
}

// =====
// Function Definition with Default Parameters
// =====
// Default values are specified in prototype above
int divide(int a, int b)
{
    int result; // Variable to store division result

    result = a / b; // Perform integer division

    return result; // Return the quotient
}

```

Expected Output:

```

divide() with no arguments:
divide() = 4

divide(12) with one argument:
divide(12) = 6

divide(63, 7) with both arguments:
divide(63, 7) = 9

```


4. CONCLUSION

This lab demonstrated advanced function concepts in C++. Call by value protects original data by passing copies, making it safe but unable to modify caller's data. Call by reference allows functions to modify original variables directly using the & operator, which is efficient for large objects and enables multiple return values. Default parameters provide flexibility by allowing functions to be called with fewer arguments, with missing parameters taking predefined values. Understanding these concepts is crucial for writing efficient, modular, and maintainable C++ programs.